

Software Architecture

Architecture Means – Basic Architecture Concepts (Component Orientation and Modularization)

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

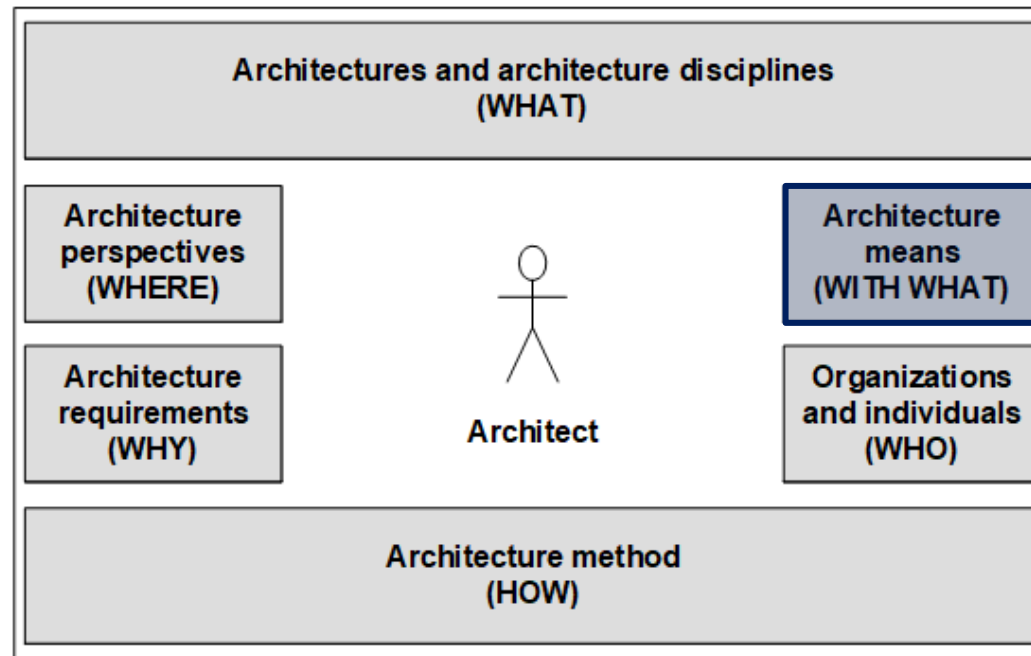
These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Lecture Opening

Architecture Orientation Framework



Architecture **WITH WHAT** discusses the broad range of architectural means architects use to plan, develop, and operate architectures. Thus what we consider here is the architect's toolbox, if you wish. [Vogel, Arnold et al 2011].



Lecture Opening

Motivation

Classes and objects are the **atoms of object-oriented programming**. Architectures, whose components are classes or objects, remain accordingly fine-grained.

With larger systems the number of the components grows substantially and the software architecture becomes accordingly unclear.

In the consequence it may **become difficult to judge the suitability of an architecture regarding its coarse goals**.

Modules / components as architecture components (summarizing several classes and (partially) abstracting from the design on class level), **provide remedy here**.

Initially the software industry hoped, that the paradigm and technology of object-oriented programming could sufficiently support modular programming. However, in the meantime we learned that it needs more.

Note that I will **use the terms module and component**, interchangeably.

Lecture Opening

Learning Objectives

You ...

- can **explain basic differences** between classes, objects and modules / components.
- can **describe basic scenarios** for the utilization of modules / components.
- can **apply and evaluate technical support** for modular programming.

Lecture Agenda



- Component
- Interface
- Composition
- Components in Java

Component Classes & objects

In this software architecture lecture so far **we have examined structures whose components were directly or indirectly tied to the idea of objects.**

In the **logical view we have examined objects directly** and in the **development view their blueprints (i.e., classes).**

Characteristic for objects is:

- They **have an identity**. This means that for two references one can determine whether they point to the same object or to two different objects.
- They can **contain their own state or data**, which are independent of other objects. Part of this data can be references to other objects.
- They can **offer specific activatable operations** (i.e., methods). How these operations are executed exactly is specific to the respective object.
- From a technical point of view, they **correspond to memory blocks** which are uniquely distinguishable and indivisible via their memory addresses.

The combination of these properties makes objects the atomic (i.e., not further subdivisible) units of the object-oriented paradigm.

Component Classes & objects

Characteristic for classes is:

- Classes are blueprints for similar objects.

In the **development view**, **classes are the smallest units** of the object-oriented paradigm.

From a software architecture perspective **object orientation offers one possible structuring level** (i.e., one possible granularity of architecture components).

However, **for many considerations this level is too fine-grained**.

As described earlier, there is a **direct relationship of a class-level granularity to identifiable storage units**.

This **makes it difficult to coarsen the structure**, e.g., to more complex tasks, responsibilities, or distributed execution units.

Component

Modules & components

The development view needs an approach, which permits **bundling classes into larger (and more coarse-grained) architecture components and to abstract from the concrete class structure in that bundling.**

Such **components represent the coarse grained structures** of an object-oriented system.

In the **hardware sector, modules / components have been established for a long time.** When, for example, televisions with semiconductors (e.g., transistors, diodes) became a mass product in the 1960s, manufacturers designed hardware architectures that combined a number of such semiconductors on pluggable circuit boards.

These boards took over partial tasks of the entire electronics and could be installed in different devices or exchanged individually in case of defects.

At that time, the **term module / component was established for such assemblies.**

Component

Modules & components

Whether in hardware or software, **modules / components are units that allow to abstract from and hide inner details and the associated design decisions.**

This also **makes it possible to revise these design decisions later without affecting other modules / components.**

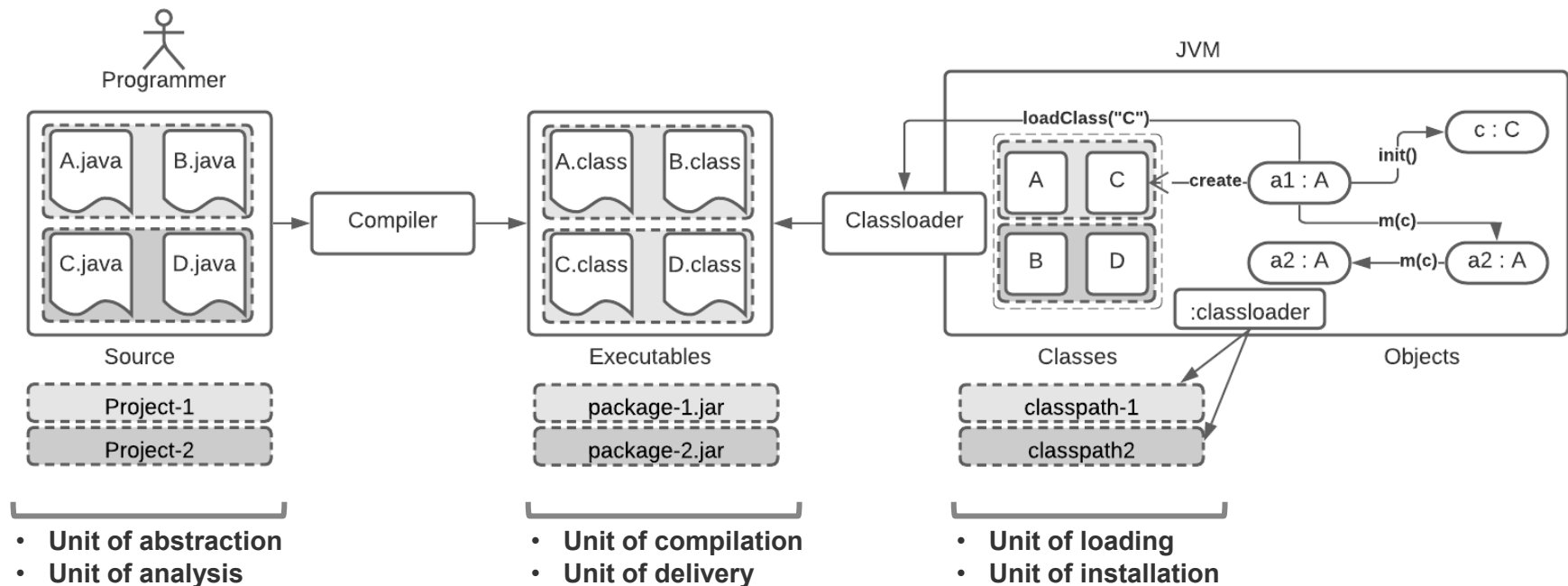
Object orientation offers the concept of information hiding – especially within classes.

Modules / components also offer information hiding, but without the restriction of having to be instantiable in memory at the same time.

The analogy of (hardware) units also shows another important property of modules / components. **One can assemble larger systems from modules / components in variants.**

Component Modules & components

Components are blueprints for related classes, while classes are blueprints for related objects.



Component Modules & components

Six properties of components in particular are important for their role as software architect.

- **Unit of abstraction.** Components hide information by hiding details of the implementation and abstracting from these details. The interface of a component is an abstraction of its actual code. **To understand what a module does (but not how it does it), it is sufficient to understand its interface.**
- **Unit of analysis.** Programmers of a component know their implementation completely. They are able to **analyze, understand and evaluate a component's internal logic**, programmed processes and their characteristics. The information Hiding hence excludes programmers of other components.
- **Unit of compilation.** In order to protect the Unit of Abstraction and the Unit of Analysis not only by conventions and discipline, but also by technical measures, all parts of the module must be present at compilation-time and must be able to be checked for consistency. Compilation creates a complete and coherent package for this module. The unit of compilation thus **ensures that the information of a module available in the source is consistently transferred to the world of deployable units.**
- **Unit of delivery.** **Modules are delivered as wholes and not in separate parts** – e.g., in parts that could possibly be assembled incorrectly or incompletely. The chosen component granularity should also make sense. This would not be the case if two different modules could only be used in this exact combination of two. So there should be at least one relevant constellation in which only one of the two modules is needed. A unit of delivery might also be linked to commercial aspects, such as price scheme, or license commitment. Also this can result in requirements for a modular architecture.

Component

Modules & components

Six properties of components in particular are important for their role as software architect.

- **Unit of installation.** Components represent the level of granularity at which decisions are made about which components to use in a system at runtime. Components can be added to a system (or omitted from a system) as a whole. This particularly also applies to different versions of a single component. All this ensures that the unit of abstraction and the unit of analysis are transferred consistently into an executable software system.
- **Unit of loading.** To ensure that object structures are generated consistently at runtime, one must ensure that the same "executables" are used as the compiler had generated them together. The Unit of Loading is therefore the last link in a chain, which is to ensure that a running system from objects behaves also exactly, as intended by the programmer.

Component

Modules & components

Although the module / component represents a central entity with the design of software architecture, **there is no uniform, or generally accepted definition.**

From numerous authors, who tried themselves at **component definitions**, I list only three.

- As per **Clemens Szyperski, Dominik Gruntz and Stephan Murer** [Szyperski et al 2002] “a software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties.”
- Following **Desmond Francis D'Souza and Alan Cameron Wills** [D'Souza and Wills 1999] “a component is a coherent package of software implementations that ...
 - has explicit and well-specified interfaces for services it provides
 - has explicit and well-specified interfaces for services it expects, and
 - can be composed with other components, perhaps customizing some of their properties without modifying the components themselves.”

Component

Modules & components

From numerous authors, who tried themselves at **component definitions**, I list only three (continued).

- The **UML 2.0 specification** defines component as a modular unit with well defined interfaces that is replaceable within its environment. [...] It has one or more provided and required interfaces, and its internals are hidden and inaccessible other than as provided by its interfaces.

Component

Modules & components

From the component definitions we can extract the following characteristics:

- A component **bundles related classes and possibly further resources**.
- A component can be **developed in separate projects under separate responsibility**.
- A component can be **distributed as a whole** (e.g. in a file) and integrated into systems, or composed into systems.
- Components allow to **separate architecture-relevant design** (i.e., design that is visible beyond the boundaries of a module / component) **from non-architecture-relevant design** (i.e., design that the architecture module / component abstracts from).
- A component **exports one or more interfaces** that are guaranteed in terms of a contract. This includes the exact semantics of the interfaces. Any component that exports an interface S (i.e., providing the operations promised in the interface) is an implementation of S.
- A **component imports other interfaces**. Importing an interface means that the component uses the methods of that interface. The component is not executable until all imported interfaces are available. This is the task of the configuration. The formulation C imports interface S leaves open who implements S.

Component

Modules & components

From the component definitions we can extract the following characteristics:

- A component **hides its implementation** and can therefore be replaced by another component that exports the same interface.
- Components are **suitable as a unit of reuse** because it makes minimal assumptions about the environment in which it will be used.
- Components can **contain other components** and can themselves compose other components.

Component

Modules & components

For component production we can distinguish (as with classes) **Singleton** and **Multiton** components. Singleton components exist as a single copy, multiton components in several.

Components usually **provide a factory** through which their users create corresponding component objects.

```
public interface ComponentObject {  
    public void method1();  
    public void method2();  
}
```

```
public class ComponentObjectImpl implements ComponentObject {  
    public void method1() {}  
    public void method2() {}  
}
```

```
public class ComponentFactory {  
    public ComponentObject getComponentObject() {  
        ComponentObject result = new ComponentObjectImpl();  
        return result;  
    }  
}
```

Lecture Agenda



- Component
- Interface
- Composition
- Components in Java

Interface Overview

Software architecture is always about the same kind of considerations: **How can a whole be broken down into smaller parts (components)** in such a way that these can be designed, programmed and developed in detail as independently as possible.

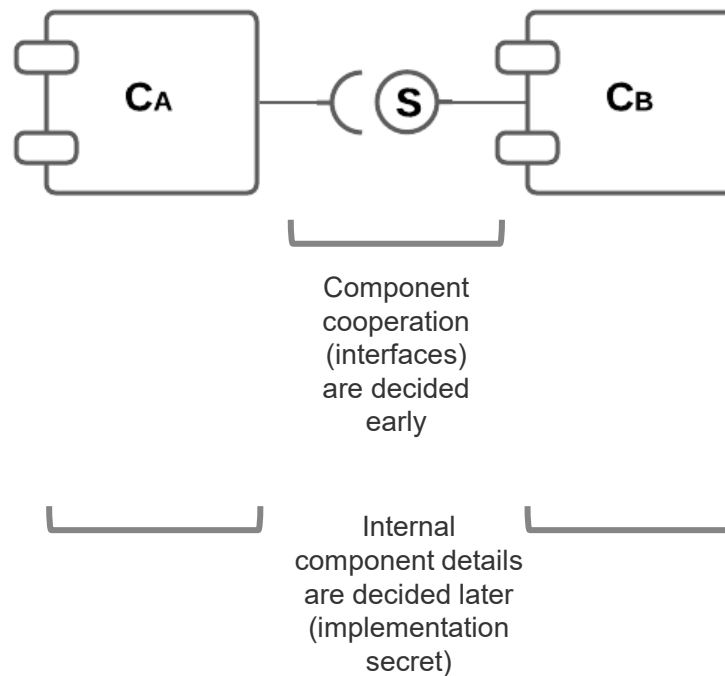
In addition, these components should cooperate meaningfully when they are put together – meaningfully in the sense that the parts realize the desired whole.

Hence, the **software architecture must precisely define what individual components offer in order to cooperate as intended.**

Interfaces are the means of expressing what a component offers. Interfaces thus define how components cooperate.

Interface Overview

This means that **you decide how several components cooperate, very early**, while you can **keep decisions about how individual components are internally structured open for later**.



Interface Overview

This section introduces a **general notion of interface**, of which the Java interface is a special case. The focus here is on the programming interface on components.

Each **programming interface defines a set of methods**, where the methods in turn have the following properties:

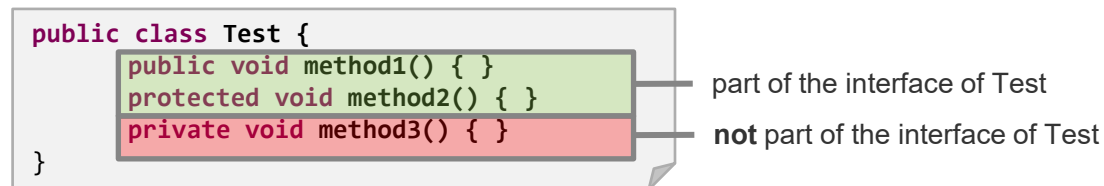
- **Syntax** (i.e., return values, types, arguments, in / out, types).
- **Semantics** (i.e., what does the method do)
- **Protocol** (e.g., synchronous, asynchronous)
- **Non-functional characteristics** (e.g., performance, availability, scalability, security)

There are dedicated **specification languages for programming interfaces** that specify both their syntax and semantics.

Interface Overview

A very simple syntactic view of program code gives us a short answer to the question “what belongs to the interface of a component?”. The answer is “everything that is accessible outside this component”.

For example, consider methods of a class declared as public. While `method1()` and `method2()` are visible and accessible from the outside, `method3()` is declared private – thus does not belong to the interface of class `Test`.

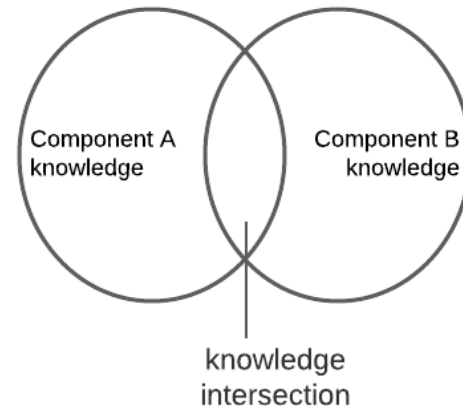
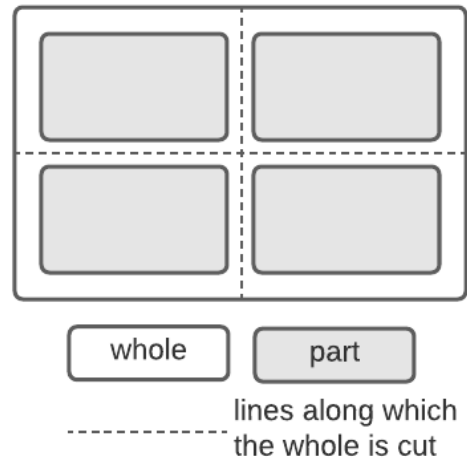


```
public class Test {  
    public void method1() { }  
    protected void method2() { }  
    private void method3() { }  
}
```

The diagram shows the code for class `Test`. The methods are highlighted with colored boxes: `method1()` is in a light green box, `method2()` is in a light yellow box, and `method3()` is in a light red box. Lines point from these boxes to text on the right: "part of the interface of Test" points to the green box, and "not part of the interface of Test" points to the red box.

Interface Overview

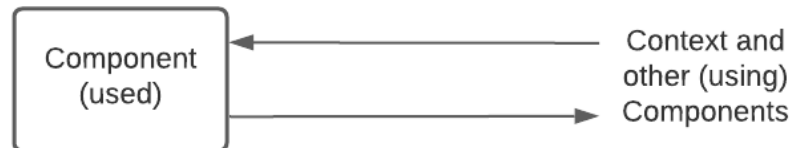
The German-language term “Schnittstelle” can be conveniently conveyed by two different images.



Interface Overview

By the **interface of a component we mean everything you need to know about a component in order to use it** (i.e., to connect it to other components), or to be able to program other components.

This includes **assumptions another component makes about a used component** but also **assumptions a used component makes about its using components and context**.



Interface Overview

What assumptions are made about `binarySearch()` and what assumptions does `binarySearch()` make about its calling context?

```
public static int binarySearch(Object[] a, Object key);
```

- Assumptions made about `binarySearch()`:
 - `binarySearch` searches for the index where `key` occurs in `a` or for the index where `key` would have to be inserted if `key` does not occur in `a`.
 - `a` is not changed
- Assumptions `binarySearch()` makes about its calling context:
 - `a` is sorted into ascending order according to the natural ordering of its elements (as by the `sort(Object[])` method).

[https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Arrays.html#binarySearch\(java.lang.Object%5B%5D,java.lang.Object\)](https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Arrays.html#binarySearch(java.lang.Object%5B%5D,java.lang.Object))

Interface Overview

In Java, a component programming interface typically consists of

- one or more Java interfaces (e.g., `public interface MyInterface { .. }`)
- all classes that are needed to address the corresponding Java interfaces. In particular, exception classes and parameter types. This part is called the interface wrapper.
- other required resources such as configuration or descriptor files insofar as they are intended for a user of the interface to modify it.

In the simplest case, a programming interface in Java is reduced to a single Java interface or Java class.

Each **component exports at least one interface** (otherwise it would not be usable) and **imports none or several interfaces**. Only interfaces are imported – not components.

An importing component makes assumptions only about the interface to be called – not about its implementation.

Interface Overview

Hence, the **exchangeability of components is a natural given** (at least in principle). However an importing component can expect non-functional characteristics from an implementing component (e.g., performance, security, availability, scalability), which can limit component exchangeability.

Interfaces, like components, are a separate unit of design and thus exist in isolation from their implementing components.

You strive to **design suitable interfaces first and only then look for suitable implementations**.

Two types of interface methods are distinguished:

- Commands change the state of a component.
- Queries leave the component state unchanged.

Interface Overview

The **semantics of an interface determines what the commands do and what queries mean.**

Degrees of freedom are often included in the interface semantics. Via the refinement of an interface you strive to limit these degrees of freedom as much as possible.

Interface Overview

The methods of object-oriented and modular design have shown how you traceably (i.e., justifiably) define which components are needed and how they should work together.

The **result of this systematic approach was interface definitions in the form of method signatures** (e.g., the signature of `binarySearch()` in the above example). In practice this is often considered as sufficient.

Strictly speaking, however, **additional information is required** (even though it might be considered as implicitly given) – very much so this is the case, when interfaces are designed for external developers.

Interface Overview

Together with the syntax level, this results in three aspects which an interface must specify or otherwise adequately address.

- Connection technology
- Syntax
- Semantics level



example

Interface Overview

Give examples for each of the three interface aspects below for the `sin()` function.

```
public double sin(double x);
```




example

Interface Overview

Give examples for each of the three interface aspects below for the `sin()` function.

```
public double sin(double x);
```

- **Connection technology**

- Answers the question about how operations are **activated**, parameters are passed and results are received.
- Examples are Java, .Net, JSON, SOAP, CORBA.

- **Syntax**

- Answers the question about how operations and parameters are **identified**.
- Java interfaces define method names, sequence of parameters, or exception and return types.
- URLs define resource addresses, names, or XML grammars.

- **Semantics level**

- Answers the question about what operations, parameters or results **actually mean**, or which requests are allowed under what preconditions.
- Parameter `x` is in radians



example

Interface Overview

Describe the semantics of the following Java interface. Pay specific attention to degrees of freedom which still exist.

```
public interface Persistence {  
    public void store(Object key, Object value);  
    public Object lookup(Object key);  
}
```



example

Interface Overview

Describe the semantics of the following Java interface. Pay specific attention to degrees of freedom which still exist.

```
public interface Persistence {  
    public void store(Object key, Object value);  
    public Object lookup(Object key);  
}
```

- **Semantics at its core is:** Any object (value) stored with store() under a key can be retrieved and restored via lookup().
- However, a variety of **degrees of freedom exist** – what remains unanswered are questions like ...
 - Are objects stored in main memory or persisted for a longer time (e.g., on disk)?
 - Which errors are possible?
 - What happens if several users store different objects under the same key?
 - Is transaction safety guaranteed (i.e., ACID principles)?

Interface Overview

If interfaces are offered in the form of publicly accessible elements (e.g., public class, interface, or method), you need to answer the question, **with which component(s) these interfaces are bundled and supplied.**

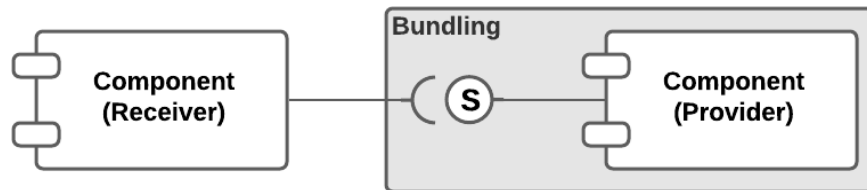
Johannes Siedersleben [Siedersleben 2004] presents an **interface-bundling classification based on three categories.**

Siedersleben assumes that there is a component, which implements the functionality and one, which using it. **Defining for his categories is, with which of these components the interface is supplied.**

Interface Overview

Provided Interface (application programming interface). Interface S is provided by the component that implements the functionality offered via S.

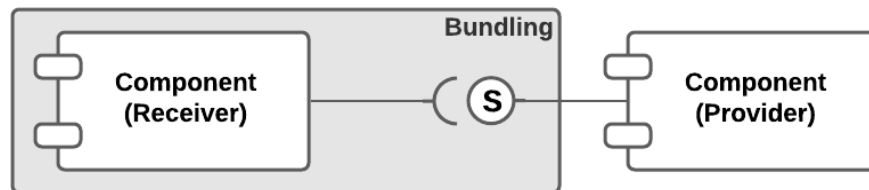
Example. Library methods.



Interface Overview

Requested Interface (service programming interface). An implementation of the interface S is requested by the component that receives via S functionality of an external component.

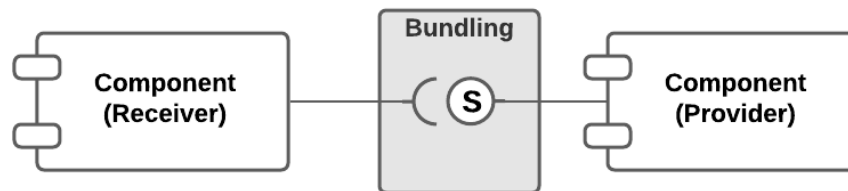
Example. An API tells you what a specific class or method does for you while the SPI tells you what you must do to conform. API and SPI are usually separate. For example, in JDBC the `Driver` class is part of the SPI. Everyone who implements a JDBC `driver` must implement that class. Sometimes API and SPI overlap, however. The JDBC `Connection` interface is both SPI and API. You use it routinely when you use a JDBC driver and it also needs to be implemented by the developer of the JDBC driver.



Interface Overview

Standard Interface. Interface S is defined in a separate component as an independent standard – independent of concrete callers and implementations.

Example. Java standards like List or Iterator. JSE provides implementations (e.g., ArrayList) but you can also introduce your own List implementations.





example

Interface Overview

In practice interface implementations are often mixed or joint responsibilities between interface requesting and providing component.

Let us look at an application of the observer pattern where requesting and providing interface definitions are combined (Observable and Observer).

```
public class Observable {  
    private List<Observer> observers = new ArrayList<Observer>();  
  
    interface Observer {  
        void update();  
    }  
    public void subscribe(Observer obs) {  
        System.out.println("Observer: implementation of subscribe()");  
        observers.add(obs);  
    }  
    public void stateChange() {  
        for(Observer o : observers) {  
            o.update();  
        }  
    }  
}
```




example

Interface Overview

Let us look at an application of the observer pattern where requesting and providing interface definitions are combined (Observable and Observer).

```
public class Client implements Observable.Observer {  
    public void update() {  
        System.out.println("Client: implementation of update()");  
    }  
}
```

```
public class Driver {  
    public static void main(String[] args) {  
        Client c1 = new Client();  
        Client c2 = new Client();  
  
        Observable o = new Observable();  
  
        o.subscribe(c1);  
        o.subscribe(c2);  
        o.stateChange();  
    }  
}
```

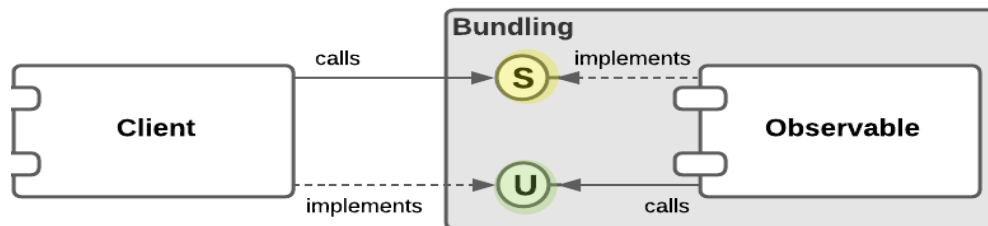


example

Interface Overview

Let us look at an application of the observer pattern where requesting and providing interface definitions are combined (Observable and Observer).

S (subscribe) – provided interface



U (update) – requested interface



example

Interface Overview

Let us finally look at an application of the observer pattern, where the interface definition is decoupled from either of the implementing components (i.e., standard interface)

```
public interface Observer {  
    public void update(String message);  
}
```

```
public class Client implements Observer {  
    public void update() {  
        System.out.println("Client: implementation of update()");  
    }  
}
```

```
public interface Observable {  
    public void subscribe(Observer o);  
}
```

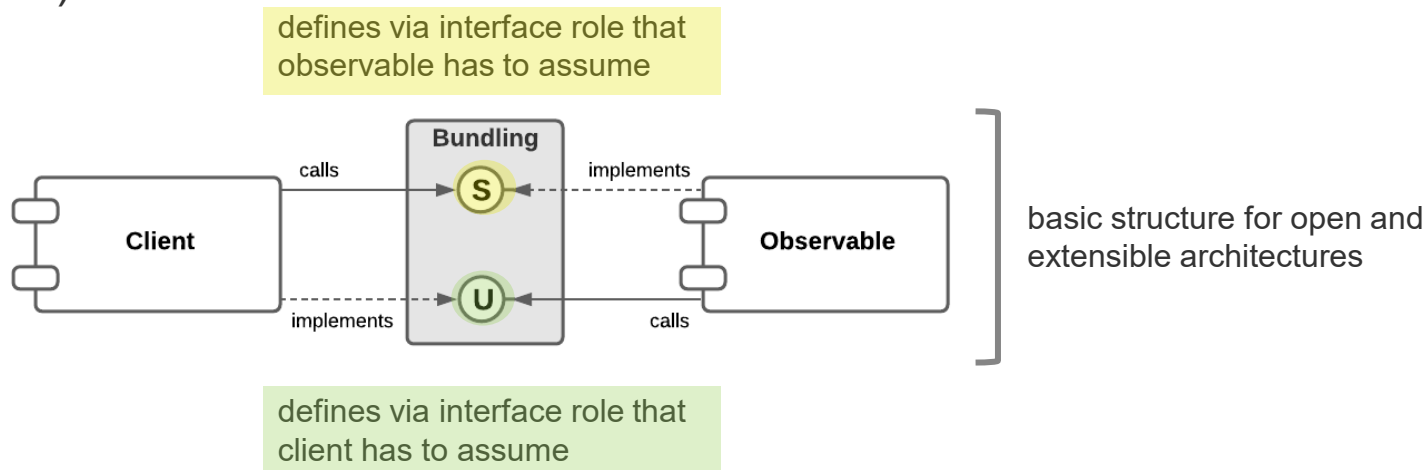
```
public class ObservableImpl implements Observable {  
    private List<Observer> observers = new ArrayList<Observer>();  
  
    public void subscribe(Observer obs) {  
        System.out.println("Observer: implementation of subscribe()");  
        observers.add(obs);  
    }  
  
    public void stateChange() {  
        for(Observer o : observers) {  
            o.update();  
        }  
    }  
}
```



example

Interface Overview

Let us finally look at an application of the observer pattern, where the interface definition is decoupled from either of the implementing components (i.e., standard interface)



Lecture Agenda



- Component
- Interface
- Composition
- Components in Java

Composition

Overview

Components decompose an overall system into manageable parts.

However, the question remains, **who will assemble these parts into an executable whole.**

For the production of a component you need implementations for each imported interface. Design and programming against interfaces are all good. However, at a certain point you must commit yourself regarding their implementation.

This happens in composition – at compile-time at the earliest, or at execution-time at the latest.



example

Composition

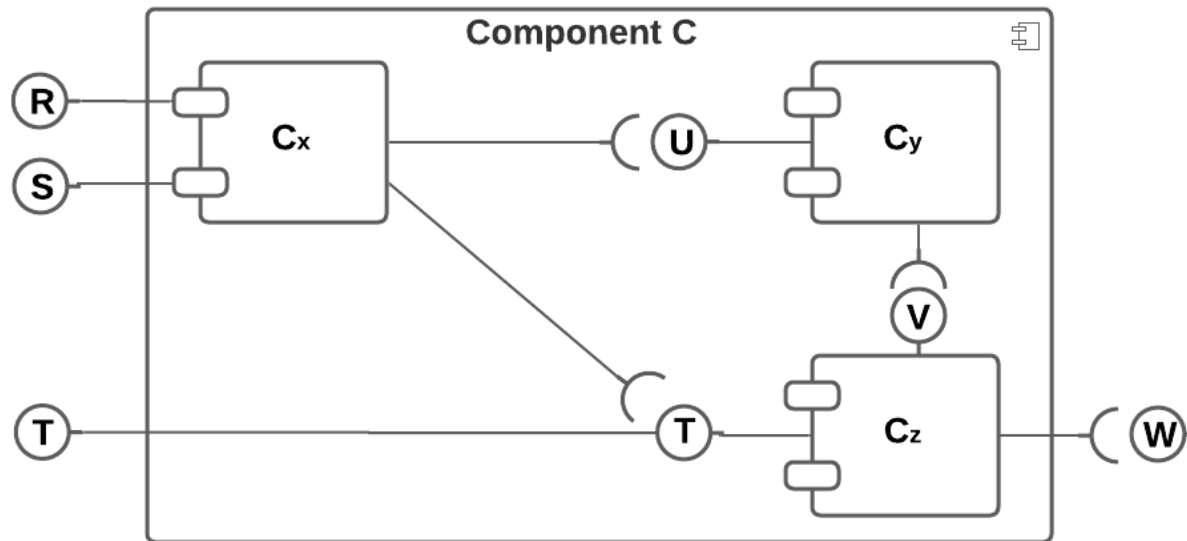
Example

Component C is comprised of three components C_x, C_y, and C_z.

C_x exports interfaces R and S, while C_z exports interface T and C_y interface U.

C_x imports interfaces U and T, while C_y imports V and C_z an external interface W.

Component C exports the interfaces R, S, and T.





example

Composition

Example

For illustration purposes, each of the interfaces is only given a single identically-named method (e.g., the interface R is given the method r()).

```
public interface R {  
    public void r(int i);  
}
```

```
public interface V {  
    public void v(int i);  
}
```

```
public interface S {  
    public void s(int i);  
}
```

```
public interface W {  
    public void w(int i);  
}
```

```
public interface T {  
    public void t(int i);  
}
```

```
public interface U {  
    public void u(int i);  
}
```




example

Composition

Example

Three classes implement all interfaces of the C component.

```
public class Cx implements R, S {
    private U u; private T t;
    public void bindU(U u) { this.u = u; }
    public void bindT(T t) { this.t = t; }
    public static Cx getCx() { return new Cx(); }
    // interface implementation
    public void r(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing r()");
        u.u(i+1);
        t.t(i+1);
    }
    public void s(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing s()");
        u.u(i+1);
        t.t(i+1);
    }
}
```

```
public class Cy implements U {
    private V v;
    public void bindV(V v) { this.v = v; }
    public static Cy getCy() { return new Cy(); }
    // interface implementation
    public void u(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing u()");
        v.v(i);
    }
}
```

```
public class Cz implements T, V {
    private W w;
    public void bindW(W w) { this.w = w; }
    public static Cz getCz() { return new Cz(); }
    // interface implementation
    public void t(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing t()");
        w.w(i+1);
    }
    public void v(int i) {
        System.out.println("executing v()");
        w.w(i+1);
    }
}
```



example

Composition

Example

One class implements the external interface W.

```
public class ComponentW implements W {  
    public static ComponentW getInstance() {  
        ComponentW m = new ComponentW();  
        return m;  
    }  
    // interface implementation  
    public void w(int i) {  
        Tabulator.printTabs(i);  
        System.out.println("executing w()");  
    }  
}
```



example

Composition

Example

Component C composes itself from all the aforementioned building blocks.

```
public class ComponentC {
    private Cx x;
    private Cy y;
    private Cz z;

    private ComponentC() {
        x = Cx.getCx();
        y = Cy.getCy();
        z = Cz.getCz();
        y.bindV(z);
        x.bindU(y);
        x.bindT(z);
    }
    public void bindW(W w) {
        z.bindW(w);
    }
    public R getR() { return x; }
    public S getS() { return x; }
    public T getT() { return z; }

    public static ComponentC getInstance() {
        ComponentC m = new ComponentC();
        return m;
    }
}
```



example

Composition Example

Finally, the use of the C component is represented by the Driver class.

```
public class Driver {  
    public static void main(String[] args) {  
        ComponentC c = ComponentC.getInstance();  
        ComponentW w = ComponentW.getInstance();  
  
        c.bindW(w);  
        R r = c.getR();  
        S s = c.getS();  
        T t = c.getT();  
  
        r.r(0);  
        s.s(0);  
        t.t(0);  
    }  
}
```

```
executing r()  
    executing u()  
        executing v()  
            executing w()  
    executing t()  
        executing w()  
executing s()  
    executing u()  
        executing v()  
            executing w()  
    executing t()  
        executing w()  
executing t()  
    executing w()
```

Composition

Façade Pattern

What types of problems does the Façade design pattern address? What is the approach of the façade pattern in dealing with these problems?

Composition

Façade Pattern

What types of problems does the Façade design pattern address? What is the approach of the façade pattern in dealing with these problems?

■ Intent

- Façade is a structural design pattern that provides a simplified interface to a library, a framework, or any other complex set of classes.

■ Solution

- A facade is a class that provides a simple interface to a complex subsystem which contains lots of moving parts.
- A facade might provide limited functionality in comparison to working with the subsystem directly.
- However, it includes only those features that clients really care about.

■ Known uses

- Layered systems (architecture pattern) use façades to decouple layers from another.
- Every pattern language is a façade to its constituent patterns.
- Façades can be stateful and stateless where state can range from conversational to persistent.
- Façades can mediate between different paradigms (e.g. procedural vs. relational vs. object-oriented). A typical example of this are ORM (object-relational-mapping) frameworks.

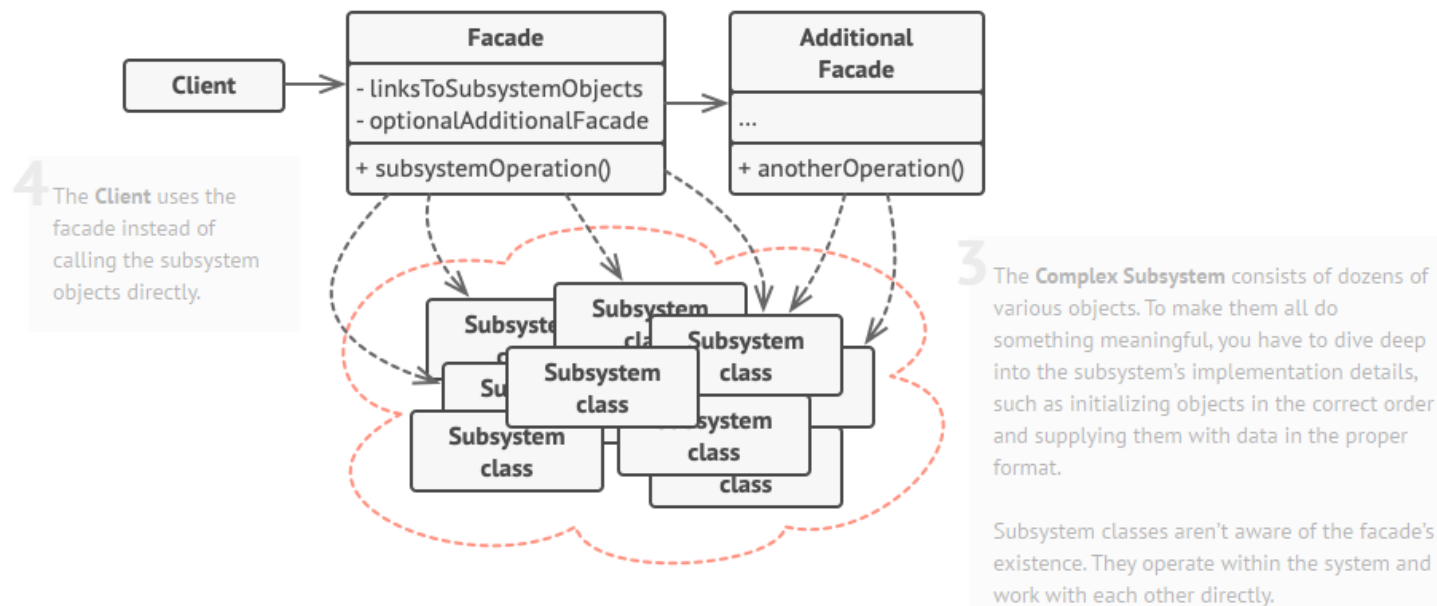
Composition

Façade Pattern

What types of problems does the Façade design pattern address? What is the approach of the façade pattern in dealing with these problems?

1 The **Facade** provides convenient access to a particular part of the subsystem's functionality. It knows where to direct the client's request and how to operate all the moving parts.

2 An **Additional Facade** class can be created to prevent polluting a single facade with unrelated features that might make it yet another complex structure. Additional facades can be used by both clients and other facades.



4 The **Client** uses the facade instead of calling the subsystem objects directly.

3 The **Complex Subsystem** consists of dozens of various objects. To make them all do something meaningful, you have to dive deep into the subsystem's implementation details, such as initializing objects in the correct order and supplying them with data in the proper format.

Subsystem classes aren't aware of the facade's existence. They operate within the system and work with each other directly.



example

Composition

Façade Pattern

Introduce a new interface U to component C and do not export R, S and T anymore. U in turn is supposed to use R, S and T in the sense of a façade.

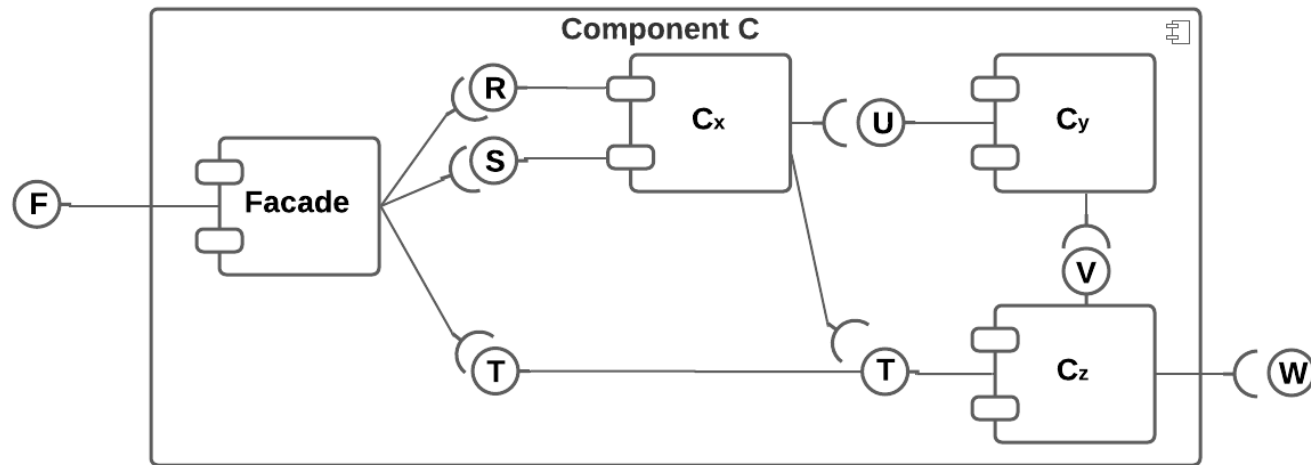


example

Composition

Façade Pattern

Introduce a new interface U to component C and do not export R, S and T anymore. U in turn is supposed to use R, S and T in the sense of a façade.





example

Composition

Façade Pattern

Introduce a new interface U to component C and do not export R, S and T anymore. U in turn is supposed to use R, S and T in the sense of a façade.

```
public interface R {  
    public void r(int i);  
}
```

```
public interface V {  
    public void v(int i);  
}
```

```
public interface S {  
    public void s(int i);  
}
```

```
public interface W {  
    public void w(int i);  
}
```

```
public interface T {  
    public void t(int i);  
}
```

```
public interface F {  
    public void f(int i);  
}
```

Façade interface

```
public interface U {  
    public void u(int i);  
}
```



example

Composition

Façade Pattern

The three classes implementing all component C interfaces remain the same.

```
public class Cx implements R, S {
    private U u; private T t;
    public void bindU(U u) { this.u = u; }
    public void bindT(T t) { this.t = t; }
    public static Cx getCx() { return new Cx(); }
    // interface implementation
    public void r(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing r()");
        u.u(i+1);
        t.t(i+1);
    }
    public void s(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing s()");
        u.u(i+1);
        t.t(i+1);
    }
}
```

```
public class Cy implements U {
    private V v;
    public void bindV(V v) { this.v = v; }
    public static Cy getCy() { return new Cy(); }
    // interface implementation
    public void u(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing u()");
        v.v(i);
    }
}
```

```
public class Cz implements T, V {
    private W w;
    public void bindW(W w) { this.w = w; }
    public static Cz getCz() { return new Cz(); }
    // interface implementation
    public void t(int i) {
        Tabulator.printTabs(i);
        System.out.println("executing t()");
        w.w(i+1);
    }
    public void v(int i) {
        System.out.println("executing v()");
        w.w(i+1);
    }
}
```



example

Composition

Façade Pattern

The same applies to the class implementing the external interface W.

```
public class ComponentW implements W {  
    public static ComponentW getInstance() {  
        ComponentW m = new ComponentW();  
        return m;  
    }  
    // interface implementation  
    public void w(int i) {  
        Tabulator.printTabs(i);  
        System.out.println("executing w()");  
    }  
}
```



example

Composition

Façade Pattern

Composition of component C changes, however.

```
public class ComponentC {
    private Cx x; private Cy y; private Cz z;
    private Facade facade;

    private ComponentC() {
        x = Cx.getCx();
        y = Cy.getCy();
        z = Cz.getCz();
        y.bindV(z);
        x.bindU(y);
        x.bindT(z);

        facade = Facade.getFacade();
        facade.bindR(x);
        facade.bindS(x);
        facade.bindT(z);
    }
    public void bindW(W w) { z.bindW(w); }
    public F getF() { return facade; }

    public static ComponentC getInstance() {
        ComponentC m = new ComponentC();
        return m;
    }
}
```



example

Composition

Façade Pattern

Next, the Façade class is introduced which implements interface F.

```
public class Facade implements F {  
    private R r;  
    private S s;  
    private T t;  
  
    public void bindR(R r) { this.r = r; }  
    public void bindS(S s) { this.s = s; }  
    public void bindT(T t) { this.t = t; }  
    public static Facade getFacade() { return new Facade(); }  
    public void f(int i) {  
        Tabulator.printTabs(i);  
        System.out.println("executing f() - the facade");  
        r.r(i+1);  
        s.s(i+1);  
        t.t(i+1);  
    }  
}
```



example

Composition

Façade Pattern

Finally, the use of the C component is represented by the Driver class.

```
public class Driver {  
    public static void main(String[] args) {  
        ComponentC c = ComponentC.getInstance();  
        ComponentW w = ComponentW.getInstance();  
  
        c.bindW(w);  
        F f = c.getF();  
  
        f.f(0);  
    }  
}
```

```
executing f() - the facade  
    executing r()  
        executing u()  
            executing v()  
                executing w()  
        executing t()  
            executing w()  
executing s()  
    executing u()  
        executing v()  
            executing w()  
        executing t()  
            executing w()  
executing t()  
    executing w()
```

Lecture Agenda



- Component
- Interface
- Composition
- Components in Java

Components in Java

JAR Files

Very early on, **JAR files were added to the Java programming model** to bundle related class files (and possibly other program resources).

Technically, these are Zip files, which is why the contents of JAR files can also be inspected with Zip programs.

All files created in a development project (especially all class files) can be packed in a JAR file and thus passed on compactly.

The JAR file of a module / component development project contains only the "own" results of this project (i.e., required third-party modules are referenced and obtained separately as JAR files).

Thus the independence in the development of the modules / component is guaranteed.

At the interface between programming and deployment, a JAR file thus represents a module / component in Java.

Components in Java

JAR Files

The assembly of a concrete system is performed via the class path of the Java runtime environment (i.e., JVM).

The class path specifies a sequence of file directories and JAR files, which are searched by the class loader when another class file is required.

Wrap-Up

■ Motivation for Components and Modularization

- **Classes** and **objects** are too **fine-grained** to manage **system complexity** in **large applications**.
- **Components group related classes** into **coarse-grained units**, improving structural clarity, component autonomy and evolvability.
- **Similar** to HW components, SW components ...
 - **Abstract internal design decisions**.
 - **Enable** replacement and **independent evolution**.
 - **Serve as reusable building blocks**.

■ Components – Core Concepts

- **Objects** are the **smallest indivisible units** (have identity, state, behavior).
- **Classes** are **blueprints** for **similar objects**.
- **Components** are **blueprints** for **related classes** and further **resources**.
- **Components represent the coarse-grained structures** of a **Software System**.

Wrap-Up

■ Six Key Properties of Components

- **Unit of Abstraction** – hides internal details, exposes clear interfaces.
- **Unit of Analysis** – can be developed and understood independently.
- **Unit of Compilation** – built, tested, and compiled as one coherent package.
- **Unit of Delivery** – distributed and versioned as a complete artifact.
- **Unit of Installation** – deployed or replaced as a whole (including versioning).
- **Unit of Loading** – loaded at runtime as a consistent, self-contained entity.

■ Common Definitions – Components ...

- are **independent units of composition** with **contractually specified interfaces** and **explicit context dependencies**.
- can be **deployed, reused, and replaced independently**.
- **hide internals, export provided interfaces, import required ones**.

Wrap-Up

■ Interfaces – Definition and Purpose

- **Interfaces define *how* components cooperate** while **keeping implementations hidden**.
- **Interfaces separate external collaboration from internal structure** (implementation / information hiding).
- **Interfaces** at least **specify**:
 - **Interface Syntax** – signature of methods, parameters, types, exceptions.
 - **Interface Semantics** – meaning and expected behavior of operations.
 - **Communication/Connection technology** – how communication happens (e.g., Java RMI, REST, SOAP, JSON).

■ Interface Assumptions – Interfaces involve 2 perspectives

- **What the consumer expects from the interface** (service semantics).
- **What the interface expects** from its **consumers** (preconditions).

■ Interface Design Implications

- **Interfaces** should be **designed before implementations**.
- **Importing components rely on interfaces, not implementations** which promotes flexibility.
- **Commands vs. Queries: distinguish state-changing vs. read-only Interfaces**.

Wrap-Up

■ Composition

- **Composition combines components** into **executable systems** by **binding imported interfaces** and **implementing exported ones**.
 - **Exported Interfaces** are **implemented** by the **composite's components**.
 - **Components implementing imported Interfaces** need to be **bound**.
- Binding occurs at ...
 - Compile-time.
 - Deployment-time.
 - Runtime (dynamic composition).
- **Ensures architectural modularity** and **evolvability**.

■ Façade Pattern in Context of Components

- **Intent:** Provide a simplified, unified interface to a complex subsystem.
- **Approach:** Create a façade class that delegates to multiple underlying interfaces (R, S, T).
- **Benefits:**
 - Simplifies usage of complex systems.
 - Decouples client code from subsystem internals.
 - Supports layered architectures (e.g., service façades, APIs).
- **Example:** Component C replaced multiple exposed interfaces (R, S, T) with one façade interface (F).

Questions



Bibliography

Lecture

[D'Souza and Wills 1999]

D'Souza, Desmond Francis, Wills, Alan Cameron, *Objects, Components, and Frameworks with UML*, Amsterdam, Addison-Wesley, 1999

[Liskov and Wing 1994]

Liskov, Barbara H.; Wing, Jeannette M., *A behavioural notion of subtyping*, in *ACM Transactions on Programming Languages and Systems (TOPLAS)*, Volume 16 Issue 6, Nov. 1994

[Martin et al 2003]

Martin, Robert C.; Rabaey, Jan M. ; Chandrakasan, Anantha P. ; Nikolić, Borivoje, *Agile Software Development - Principles, Patterns, and Practices*, Amsterdam, Pearson Education, 2003

[Siedersleben 2004]

Siedersleben, Johannes, *Moderne Softwarearchitektur – umsichtig planen, robust bauen mit Quasar*, Köln, Dpunkt-Verlag, 2004

Bibliography

Lecture

[Szyperski et al 2002]

Szyperski, Clemens; Gruntz, Dominik; Murer, Stephan, Component Software - Beyond Object-oriented Programming, Amsterdam, Pearson Education, 2002

[UML 2.0]

OMG, Unified Modelling Language Specification 2.0, 2004, <https://www.uml.org>

[Vogel, Arnold et al 2011]

Vogel, Oliver; Arnold, Ingo; Chughtai, Arif; Kehrer, Timo, [Software Architecture: A Comprehensive Framework and Guide for Practitioners](#), Springer Science and Business Media, Berlin Heidelberg, 2011